GESTIÓN DE CONFIGURACIÓN Y MANTENIMIENTO DE SOFTWARE · UNIDAD 3

Recorrido resuelto

Los formatos F-15 a F-20 con Flask y con la plantilla del curso · Maestría en Ingeniería de Software, Universidad de La Sabana

Este recorrido usa dos fuentes reales. **Flask**, el ejemplo del docente desde la Unidad 1, muestra cómo se ven una estrategia de ramas, una política de versiones y unas métricas de entrega en un proyecto con quince años de historia. **La plantilla del curso**, en la carpeta `plantilla/` de este repositorio, muestra el pipeline corriendo en GitHub, con una ejecución fallida y su corrección. Datos consultados el 1 de noviembre de 2026.

> CÓMO USAR ESTE RECORRIDO
>
> Su entrega se parece más a la segunda parte que a la primera: un repositorio nuevo, con poca historia, en el que todo lo que se configura se puede mostrar con un enlace.

Primera parte · Flask

F-16 · Estrategia de ramas

Estrategia	Rama de mantenimiento: `main` para la versión siguiente y `stable` para corregir la vigente.
Evidencia	`stable` está 87 commits detrás de `main` y no tiene commits propios pendientes: todo lo que entra a `stable` se integra después a `main`. En el remoto solo hay otras dos ramas, de un commit cada una: las ramas de trabajo viven poco.
Por qué le sirve	Es una biblioteca con usuarios en la versión vigente. Las correcciones de seguridad de la 3.1.1 y la 3.1.3 salieron sin esperar a la 3.2.0.

F-17 · Versionamiento y entregas

Formato	`X.Y.Z` sin prefijo. De 69 etiquetas, 47 cumplen versionamiento semántico. Las 22 restantes son las versiones antiguas de dos números (0.1 a 1.0), siete etiquetas que nombran ramas de mantenimiento (como `2.0.x`) y `2.0.0rc1` y `2.0.0rc2`, que en SemVer se escriben con guion: `2.0.0-rc1`.
Contrato	Flask no sigue SemVer estricto. La 2.3.0, una versión menor, retiró código marcado como obsoleto en la 2.2.0: bajo SemVer eso exige una versión mayor. El proyecto compensa con un período de obsolescencia de una versión.
Registro de cambios	`CHANGES.rst`, una sección por versión con fecha y referencias a PR, issues y avisos de seguridad.

Lección para su F-17: declaren qué significa cada número en su sistema. Una política distinta de SemVer estricto es válida si está escrita y se cumple.

F-20 · Métricas de entrega desde 2021

```
python herramientas/entrega.py flask --desde 2021-01-01
```

Métrica	Valor	Lectura
Versiones por año	2021: 5 · 2022: 8 · 2023: 8 · 2024: 4 · 2025: 2 · 2026: 1	La frecuencia baja desde 2024, como mostró la Unidad 1
Tipos de versión	2 mayores, 4 menores, 22 de corrección	79 % son de corrección. En una biblioteca eso no indica fallos: muchas correcciones son de tipos o de documentación
Tiempo de entrega de los cambios	Mediana 61 días · percentil 75: 172 días	Un cambio típico espera dos meses a la siguiente versión
Mensajes de commit en `main`	868 · Conventional Commits 0 % · con referencia 3 %	La convención del proyecto es otra: frases cortas en imperativo, con el PR en la fusión

Cada commit se cuenta en la primera versión que lo publicó, de cualquier rama. Sin esa regla, la 2.0.0 aparecía con 3.066 commits porque las versiones de mantenimiento 1.1.3 y 1.1.4 salieron después de ella. El detalle por versión está en `ejemplo_flask/entregas.csv`.

Segunda parte · La plantilla del curso

F-15 · Repositorio y convenciones

Estructura	`servicio/` (el sistema), `tests/`, `scripts/` (verificación de versión, notas de la entrega, prueba de humo), `.github/workflows/` (CI y entrega), `Dockerfile`, `CHANGELOG.md`.
Versión declarada	En dos lugares: `pyproject.toml` y `servicio/__init__.py`. El pipeline de entrega no publica si no coinciden con la etiqueta.
Dependencias	Fijadas con versión exacta en `requirements.txt` y `requirements-dev.txt`.
Calidad	ruff para errores comunes y formato. En la primera prueba local detectó tres líneas demasiado largas en uno de los scripts de la plantilla.

F-18 · Diseño del pipeline

Etapa	Disparador	Qué hace	Qué la hace fallar
calidad	Cada PR y cada integración a `main`	`ruff check` y `ruff format --check`	Errores de estilo o de formato
pruebas	Después de calidad	`pytest` en Python 3.11, 3.12 y 3.13; los resultados se guardan como artefacto	Una prueba que falle en cualquier versión
imagen	Después de pruebas	Construye la imagen con el commit adentro, la levanta y comprueba `/salud` y `/version`	Que no arranque, o que `/version` no devuelva el commit construido
verificar	Etiqueta `vX.Y.Z`	`scripts/verificar_version.py`	Formato inválido, versión declarada distinta o sin sección en el registro de cambios
publicar-imagen	Después de verificar y pruebas	Construye una sola vez y publica en GitHub Container Registry, con la versión y el commit como etiquetas	Falla de construcción o de publicación
staging	Después de publicar	Levanta la imagen publicada, por su digest, y corre la prueba de humo	La imagen publicada no responde o no es la del commit
produccion	Aprobación del entorno	Publica la entrega con las notas del registro de cambios y el digest de la imagen	Rechazo en la aprobación

Las acciones externas están fijadas por hash de commit, con los mismos hashes que usa Flask. Los secretos son solo el `GITHUB_TOKEN` que GitHub entrega a cada ejecución: no hay ninguno en el repositorio.

F-19 · Registro de evidencias

Las tres ejecuciones del flujo «CI de la plantilla» en el repositorio del curso, el 1 de noviembre de 2026:

#	Qué demuestra	Ejecución y commit	Resultado
1	CI en verde sobre el PR	41256889959 · `e07ec9e8`	Los cinco trabajos en verde en unos 40 segundos: calidad, pruebas en 3.11, 3.12 y 3.13, e imagen con prueba de humo
2	Una falla que bloquea un cambio	41257272061 · `20c7629f`	El cambio pone `/salud` en mayúsculas. La prueba falla en las tres versiones: `assert {'estado': 'OK'} == {'estado': 'ok'}`. El trabajo `imagen` queda omitido: nada se construye sobre un cambio que no pasa las pruebas
3	La corrección	41257593217 · `05b0e583`	El cambio se revierte y la CI vuelve a verde

Las ejecuciones se ven en la pestaña Actions del repositorio del curso. La falla fue provocada para el ejemplo; en su sistema, las fallas reales son mejores evidencias que las provocadas.

Lo que no se ejecutó

El flujo de entrega no se corrió en el repositorio del curso: hacerlo publicaría una imagen y una entrega en él. Se validó con `actionlint`, y cada pieza se probó por separado:

- `verificar_version.py` aceptó `v0.1.0` y rechazó `v0.2` con cuatro errores;
- `notas_version.py` extrajo la sección 0.1.0 del registro de cambios;
- la imagen se construyó, corrió con un usuario sin privilegios y pasó la prueba de humo, que también falló correctamente con un commit equivocado.

La primera vez que lo corran en su repositorio, guarden esa ejecución en el F-19.

Referencias

Forsgren, N., Humble, J., & Kim, G. (2018). *Accelerate: The science of lean software and DevOps*. IT Revolution Press.

Humble, J., & Farley, D. (2010). *Continuous delivery: Reliable software releases through build, test, and deployment automation*. Addison-Wesley.

Pallets. (2026). *Flask* [Repositorio de código fuente]. GitHub. https://github.com/pallets/flask

Preston-Werner, T. (s. f.). *Semantic Versioning 2.0.0*. https://semver.org

 · Licencia CC BY-NC-SA 4.0. Flask es un proyecto de Pallets con licencia BSD-3-Clause; este documento analiza su repositorio público.